

Dart Tutorial 3: Operators in Dart

This chapter explains the different types of operators available in Dart. After completing this tutorial, students will understand how to perform calculations, comparisons, and logical operations in Dart programs.

1. Introduction to Operators

An **operator** is a special symbol used to perform an operation on one or more values, called operands. Operators are essential for performing calculations, comparing values, and combining conditions in a program. Dart provides several categories of operators, each used for a specific purpose.

2. Why We Use Operators in Dart

In real world apps, we constantly need to calculate values, compare data, and combine conditions.

Operators allow us to:

- Perform mathematical calculations.
- Compare two values to make decisions.
- Combine multiple conditions together.
- Assign and update variable values.

Because of this, operators are used in almost every line of logic in a Dart program.

3. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

Operator	Description	Example
+	Addition	5 + 2 = 7
-	Subtraction	5 - 2 = 3
*	Multiplication	5 * 2 = 10
/	Division	5 / 2 = 2.5
~/	Integer Division	5 ~/ 2 = 2
%	Modulus (Remainder)	5 % 2 = 1

```
int a = 10;  
int b = 3;  
  
print(a + b);  
print(a % b);
```

Output:

```
13  
1
```

4. Relational (Comparison) Operators

Relational operators are used to compare two values. They always return a `bool` result.

Operator	Description	Example
<code>==</code>	Equal to	<code>5 == 5 → true</code>
<code>!=</code>	Not equal to	<code>5 != 3 → true</code>
<code>></code>	Greater than	<code>5 > 3 → true</code>
<code><</code>	Less than	<code>5 < 3 → false</code>
<code>>=</code>	Greater or equal	<code>5 >= 5 → true</code>
<code><=</code>	Less or equal	<code>5 <= 3 → false</code>

```
int age = 18;  
print(age >= 18);
```

Output:

```
true
```

5. Logical Operators

Logical operators are used to combine multiple conditions together.

Operator	Description	Example
&&	AND (both true)	true && false → false
	OR (any one true)	true false → true
!	NOT (reverses value)	!true → false

```
bool hasLicense = true;
int age = 20;

print(hasLicense && age >= 18);
```

Output:

```
true
```

6. Assignment Operators

Assignment operators are used to assign and update values in variables.

Operator	Description	Example
=	Assign value	x = 5
+=	Add and assign	x += 2
-=	Subtract and assign	x -= 2
*=	Multiply and assign	x *= 2
/=	Divide and assign	x /= 2

```
int score = 10;
score += 5;
print(score);
```

Output:

```
15
```

7. Increment and Decrement Operators

Dart provides shortcuts to increase or decrease a value by 1.

```
int count = 5;
count++;
print(count);

count--;
print(count);
```

Output:

```
6
5
```

8. Null Aware Operators

Dart provides special operators to safely work with null values.

Operator	Description	Example
??	Returns value if not null, otherwise default	name ?? 'Guest'
??=	Assigns value only if variable is null	name ??= 'Guest'

```
String? username;
print(username ?? 'Guest');
```

Output:

```
Guest
```

9. Conclusion of Tutorial 3

In this tutorial, students have learned:

1. What operators are and why they are used.
2. How to use Arithmetic operators for calculations.

3. How to use Relational operators for comparisons.
4. How to use Logical operators to combine conditions.
5. How to use Assignment, Increment, and Decrement operators.
6. How to safely handle null values using Null Aware operators.

In the next tutorial, we will learn about Conditional Statements in Dart.

This completes Dart Tutorial 3.